The Application of Deep Learning to Automated Waste Categorisation Using Image Classification

Louis Vidal

20/06/2025

Abstract

This project investigates how machine learning can be applied to automatic waste categorisation using image recognition. Motivated by growing concerns with the modern recycling industry and its role in poor waste management, I explored how deep learning could replace manual sorting. Throughout this project, I developed two models: a Multi-layer Perceptron (MLP) built from scratch and a Convolutional Neural Network (CNN) implemented with Pytorch, trained on the Kaggle Waste classification dataset.

The CNN achieved an accuracy of 71%, significantly outperforming the MLP with 32%. This confirms the superior architecture of CNNs for visual recognition tasks. In addition to the functional artefact, this project enhanced my understanding of neural network mathematics, data preprocessing, and model evaluation.

Introduction

The growing global waste crisis presents a significant environmental challenge, with increasing amounts of rubbish overflowing landfills and polluting ecosystems. Globally, we generate over 2 billion tons of waste a year. A major reason is that our existing recycling methods are inefficient and not suited to the scale at which the modern population produces waste. To address this issue, more efficient and scalable methods of sorting and recycling materials are urgently needed. In response, I am focusing my Extended Project Qualification on developing a machine learning model capable of classifying images of waste in real time using a live camera feed. The system sorts items into six categories: glass, metal, paper, plastic, cardboard, and general trash. I first stumbled upon this concept after reading about how deep learning models were used to classify birds in the wild. I realised that although differentiating between bird pictures might seem complex due to the few distinguishing features that birds have, some models, such as EfficientNet-B0 (Tan & Le 2019), ResNet152V2 (He et al. 2016) and DenseNet201 (Huang et al. 2017), achieved an accuracy of 99.51%(Papers With Code 2023). I realised a similar simplified approach could be implemented for environmental challenges. Initially, I planned to build a model using basic image-processing techniques. However, this led to some unusable accuracy; further details will be provided in

later sections. Further research soon revealed that CNNs were far more effective for visual recognition tasks. This encouraged me to explore machine learning frameworks such as PyTorch (PyTorch Contributors 2019), as implementing a convolutional neural network from scratch at a low level would have been significantly more complex given the scale and time constraint. This automated classification helps facilitate the recycling process by reducing contamination and improving the efficiency of the sorting. Using AI technology provides a faster, more reliable, and easily expandable way to handle classification compared to the old manual methods. Traditional sorting can be time-consuming, prone to mistakes, and costly to maintain. By cutting down on the need for people to be involved in every step, this approach not only saves money but also improves the quality of recycled materials.

This project investigates how an emerging technology, such as artificial intelligence, might enhance the efficacy of waste management systems, eventually increasing recycling sustainability and accessibility. It highlights the important role intelligent methods can play in addressing environmental issues and beating manual traditional methods in meeting global sustainability goals. This topic drew my attention because machine learning is better suited to producing accurate results even with unseen or unpredictable data than traditional algorithmic approaches that rely on rigid rules and predefined input structures. Its ability to learn patterns from data, rather than being explicitly programmed for every scenario, makes it a more flexible and powerful tool for real-world problems like waste classification. For the goal of classifying rubbish that comes in all colours, shapes, and sizes, artificial intelligence seems like the logical fit for this purpose.

Through this project, my goal goes beyond simply building a functioning model; I aim to gain a deeper understanding of how AI systems are developed, trained, and evaluated. Furthermore, I strive to gain a mathematical understanding of how relevant machine learning systems work. A foundational text in this area is "Deep Learning" by Goodfellow, Bengio, and Courville (Goodfellow et al. 2016), which provided a theoretical basis for my work. The principles of gradient-based learning, as outlined by LeCun et al. (LeCun et al. 1998), also heavily influenced my approach to training. As someone relatively new to this field, I anticipate that this project will involve ongoing learning and experimentation to optimise my final product. From selecting the appropriate model architectures to finding the right datasets, and addressing the limitations of evaluation accuracy and computational resources.

Background Research

Before building the model, I carried out background research into machine learning techniques, more specifically those used for image classification. The task of classifying waste images demands a model capable of understanding textural and spatial features within an image. Furthermore, each pixel value can only be interpreted by considering the adjacent pixel values, known as spatial dependence or spatial correlation. Each category often shares visual features such as colour, shape and backgrounds, making it essential for the model to detect subtle local patterns more accurately to differentiate between the categories. I

began by researching multi-layer perceptrons (MLPs), a type of feedforward neural network consisting of fully connected layers with nonlinear activation functions such as the ReLU (Rectified Linear Unit)(Nair & Hinton 2010) or the sigmoid activation. This more basic architecture is more commonly used for simpler projects in introductory machine learning. MLPs require input data to be flattened into a one-dimensional vector, thereby removing all two-dimensional features that are critical to classifying the image. This means that multi-layer perceptrons treat neighbouring pixels the same as much more distant pixels, where the relationship might be irrelevant, preventing them from capturing local dependencies, such as edges or textures. As a result, these basic feedforward neural networks do not generalise as well on visual tasks, particularly when faced with images that require more to learn, more intricate patterns and features, especially when classes share similar visual features. Additionally, these models often fail when faced with variation in lighting or orientation. However, for this specific implementation, the MLP can yield moderately promising results because many classes vary noticeably in colour. This allows the MLP to achieve a reasonable accuracy by primarily relying on colour information, even though it cannot learn important features such as edges or textures. For instance, an image of cardboard might have darker grey-brown tones while glass has a lighter green colour. This approach is limited when classes have overlapping colour profiles, such as plastic and glass, or general waste and paper.

For this project, I used the Kaggle Waste Classification Data dataset (asdasdasasdas 2022). It contains over 2500 images of various waste materials categorised into six classes. This data set presents a valuable challenge for image classification models as a result of its diverse collection of real-world photos captured under different lighting conditions and backgrounds. Using a publicly available dataset like this allowed me to benefit from a well-curated, labelled dataset, saving time on curating and annotating images myself. The variety of the data set also emphasised the need for a model architecture capable of effectively capturing complex visual patterns, allowing it to generalise across differences in texture, colour, and shape.

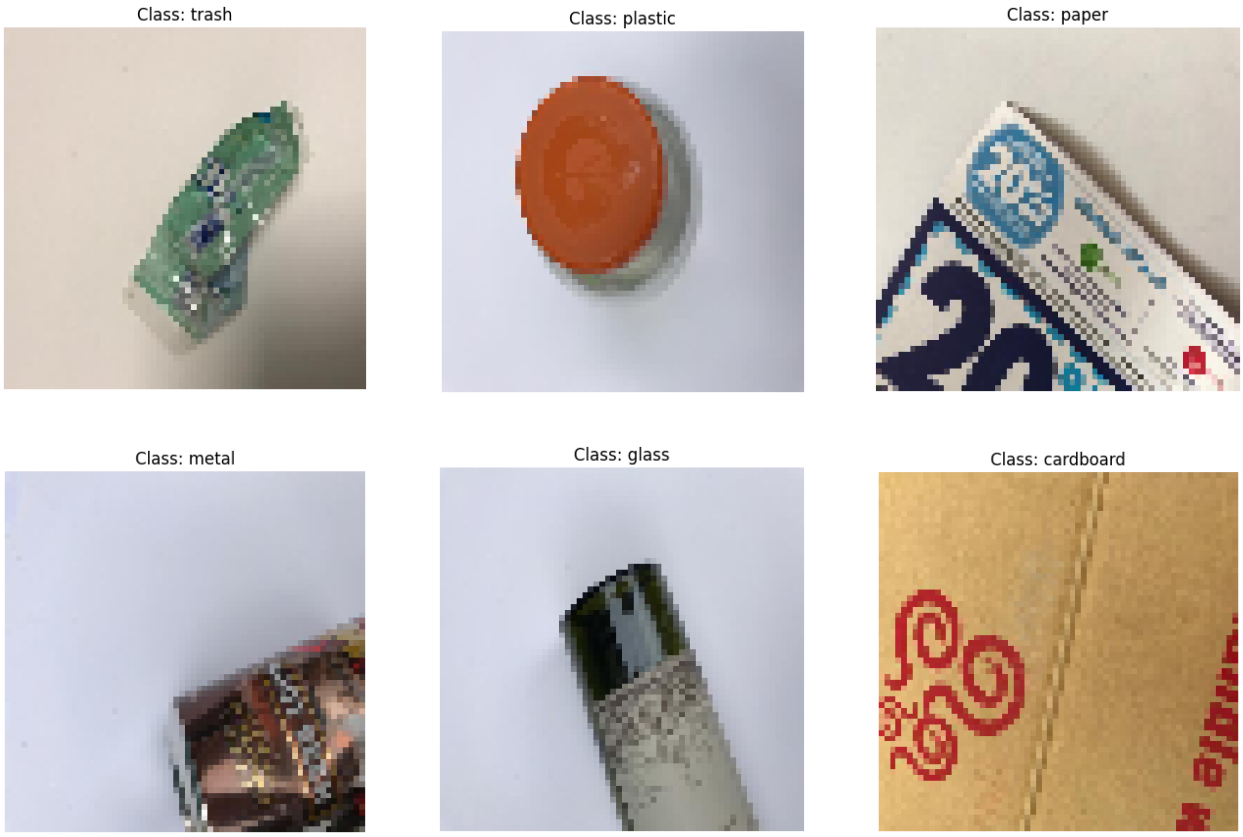

Figure 1: Example images for each category

I identified the limitations of MLPs and began exploring convolutional neural networks, which are specifically designed to handle more complex features and patterns within image data. CNNs preserve spatial hierarchies using local receptive fields, where each neuron processes only a small region of the input image at a time, and shared weights in the form of convolutional filters that capture context around the pixel. These architectural differences allow CNNs to learn more complex visual features while being relatively small compared to MLPs. Due to their proven success across many fields involving computer vision, CNNs have become the standard for image-based classification and were ultimately chosen for my core architecture for my project. The success of CNNs was notably demonstrated by models like AlexNet (Krizhevsky et al. 2012) on the ImageNet challenge. Convolutional neural networks use specialised layers called convolutional layers, which apply a small filter iteratively moving it across the input to detect local features. This works because using the values for pixels around the centre of the kernel captures the context of the area. This works better than traditional "vanilla" layers that only process a single pixel at a time, which do not effectively capture the relationship between a pixel and the surrounding region. Following convolution, pooling layers are used to reduce the dimensions by summarising regions (by taking averages or using the maximum value), which helps to reduce the computation and make the model generalise more efficiently, in turn making it more robust to smaller changes or distortions in the input. Together, this set of layers allows CNN to learn hierarchical feature representation, which makes it highly effective for image classification.

Planning and Development

The development phase of my project began with the necessary preprocessing of my data. After some research on image input formats for neural networks, I decided to resize each image to a fixed resolution of 64×64 pixels. This resolution meant a balance between maintaining enough visual detail for classification and reducing computational complexity, which was especially important given the hardware limitations that I had. A specific algorithm had to be developed to navigate through each subdirectory, treating each subfolder as a separate class label. Initially, each image was loaded with a greyscale value to reduce the input size. To make the data compatible with a multilayer perceptron, each image was then flattened into a one-dimensional vector of 4032 values (one for each pixel). This transformation allowed the images to be treated as standard numerical input, but it also meant losing the spatial structure of the original images, as discussed in the previous section. This limitation was realised only later in the project. Finally, each pixel value is normalised by dividing by 255 using min-max normalisation. This procedure is used to scale the values of the pixels to a standard range (typically 0 - 1), which helps neural networks to function more efficiently. Without this necessary process, raw pixel values ranging from 0 to 255 can cause large gradients or unstable weights when updating them through linear regression. By compressing the values, the model converges faster relative to denormalised data and becomes less sensitive to differences in input scales

```python
def load_and_preprocess_data(data_dir, img_size=(64, 64)):
    X = []
    y = []
    categories = []

    # each class has it's own folder, so we need to download each file from the respective folder
    for d in os.listdir(data_dir):
        if os.path.isdir(os.path.join(data_dir, d)) and not d.startswith('.'):
            categories.append(d)

    print("Found categories:", categories)
    categories.sort()
    # loading every category
    print("Loading and preprocessing images...")
    for category_idx, category in enumerate(categories):
        category_path = os.path.join(data_dir, category)
        if not os.path.isdir(category_path):
            continue

        print(f"Processing Category: {category} ({category_idx})")
        #getting each image from each category
        image_files = [f for f in os.listdir(category_path) if f.lower().endswith('.jpg')]

        for img_file in tqdm(image_files[:500]):
            img_path = os.path.join(category_path, img_file)
            img = cv2.imread(img_path)
            img = cv2.resize(img, img_size)
            img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            img_rgb = img_rgb / 255.0

            X.append(img_rgb)
            y.append(category_idx)

    return np.array(X), np.array(y), categories
```

Figure 2: Python file to preprocess data

The complete source code for this project can be found in the public GitHub library. siouladiv (2024)

The general formula for min-max normalisation is as follows:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

At this point, I built a deeper MLP architecture to address these limitations. The final model had four hidden layers with 2056,1028,1028, 512, 256, and 128 neurones, respectively, and an input layer of size 4032 (for the flattened RGB images). The output layer matched the number of final categories: 6. I trained this architecture using 50 epochs and a learning rate of 0.001. This lower learning rate was used so that the model could converge more smoothly as the model size increased. Despite these changes, the accuracy still plateaued at around 32%. This usually indicates limited generalisation.

The next phase of my project involved an iterative and practical approach to improve the accuracy of the model while deepening my mathematical understanding of neural networks. Committed to building a neural network from scratch, I researched the best ways of structuring my code for efficiency and ease of understanding. This led me to adopt an object-oriented approach to structure my model, which allowed me to modularise each component of the layers, such as activations, the training loop and the final prediction logic, promoting code reuse and simplicity. I used numerous tutorials and articles to build my understanding; however, it was the detailed tutorial by EL Caiseri (Caiseri 2022) that served as my key reference throughout this process. This tutorial offered a clear and practical framework for implementing an MLP using NumPy while also deepening my understanding of the mathematical principles behind each component. I followed a modular approach as suggested in the tutorial.

Building on this, I implemented an MLP from scratch to better comprehend how data flows through a neural network. To aid understanding, it's helpful to briefly outline how this architecture is structured. An MLP theoretically acts as a universal approximator, capable of modelling any continuous function given sufficient complexity and training time. A multilayer perceptron consists of an input layer, one or more hidden layers and an output layer. Each layer of nodes is connected to every node of the next layer, forming a familiar net that you might have seen before.
(ResearchGate 2024)

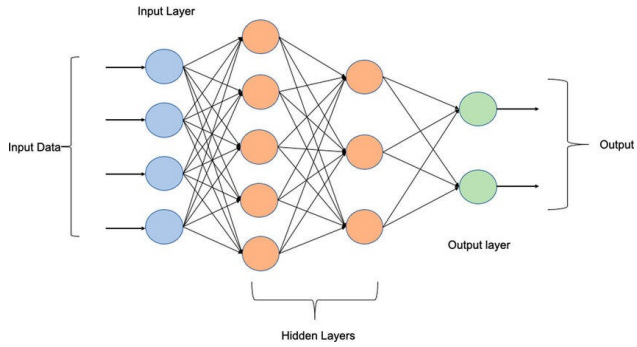

Figure 3: MLP simplified Diagram

Each node in the input layer holds a pixel value from our images, meaning the size of the input layer is determined by the cardinality of the input data. In this case, the size of the input layer is 4032. Every connection between neurons in adjacent layers corresponds to a learnable weight, which determines how strongly each input influences the next layer during forward propagation. I used the ReLU (Rectified Linear Unit) activation function in the hidden layers because it introduces non-linearity, enabling the neural network to learn and represent much more complex patterns than would be possible with solely linear layers.

Unlike sigmoid or tanh activations, which squash output into a limited range and can suffer from vanishing gradients, ReLU outputs zero for negative results and a linear response for positive results. This can be modelled by the max function, max(0,x). This characteristic often improves convergence by a factor of 6 in deep learning models. However, ReLU may cause some neurons to remain inactive if they output zero consistently.

The next important step is forward propagation, where input data is passed through the network layer by layer. Next, during forward propagation, inputs are multiplied by weights and summed with biases, and passed through an activation function to produce an output. This transforms raw pixel data into predictions. Following this, backpropagation calculates the error between predictions and true labels, then works through the network to update the weights using calculus. This process allows the MLP to learn from its output and iteratively modify its weights to produce a better output.

The MLP class I developed contains three key components: initialisation, forward propagation, and backpropagation. This implementation gave me hands-on insight into how neural networks learn from data, and it prepared me for the more complex CNN model I would later build using PyTorch(PyTorch Contributors 2019).

Model Evaluation and Reflection

The dataset images are divided into two parts, 20% for the testing of the model and 80% for the training process, as evaluating a model with data it has trained on would show inaccu-

rately high results. After completing the MLP implementation, I evaluated its performance on the waste classification task with the test data. This yielded an unexpectedly low result that was comparable to randomly guessing, revealing the model's inability to capture complex patterns in the data. To improve the model's accuracy, I made several iterative adjustments to the structure of the model. I first assumed that the model wasn't large enough, meaning there were not enough hidden layers to capture more complex features of the images. To account for this, I first increased the number of hidden layers and expanded each layer to include more neurons. This gave the model a greater learning capacity and produced a slight improvement, but not enough to be considered useful. Next, I increased the image resolution to 128×128 pixels to retain more visual detail. Finally, I introduced colour images instead of greyscale, allowing the model to make use of differences in RGB channels. These changes collectively raised the model's accuracy to a maximum of 31%.

Despite this improvement, increasing the model any further lead to over-fitting. Validation accuracy plateaued meaning that the model was memorising the training data without generalising properly. This changed my perspective where I realised the importance of balancing model complexity with generalisation avoiding increasing the size of the model unnecessarily where there might be over-fitting. Realising the limitations of the MLP, I transitioned to implementing a convolutional neural network (CNN) using the Pytorch framework. CNNs are better suited for classifying image data, as discussed earlier. Pytorch provided a flexible framework for designing and training the CNN with built-in support for CUDA drivers. This enables me to leverage an Nvidia graphics processing unit for significantly faster training. This shift marked a significant step forward in the project, allowing for better accuracy and scalability in the waste classification task.

```python
class CNN(nn.Module):
    def __init__(self, num_classes=6):
        super(CNN, self).__init__()

        self.conv_layers = nn.Sequential(
            # Block 1: Input (3, 64, 64) Output (32 , 32, 32)
            nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2), # Reduces to 32 x32
            nn.Dropout2d(0.1),

            # Block 2: Input (32, 32, 32) Output (64, 16, 16)
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2), # Reduces to 16x16
            nn.Dropout2d(0.1),

            # Block 3: Input (64, 16, 16) Output (128, 8 , 8)
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2), # Reduces to 8x8
            nn.Dropout2d(0.2),

            # Block 4: Input (128, 8, 8)  Output (256, 4, 4)
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.MaxPool2d(2), # Reduces to 4x4
            nn.Dropout2d(0.3),
        )

        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(256 * 4 * 4, 512)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(512, num_classes)
```

Figure 4: CNN architecture

To assess the impact of these changes, I tracked several performance metrics to better understand how factors like input size or the number of hidden layers affected the model. Specifically, I measured the F1-score, recall, and precision to evaluate performance beyond overall accuracy. Each score measures a different aspect of the model's output to give a more complete representation of the performance. A comprehensive review of these metrics can be found in the work by Powers (Powers 2011). The accuracy of the model can be calculated with the following equation:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

TP (True Positives): Correctly predicted positive cases. **TN (True Negatives):** Correctly predicted negative cases. **FP (False Positives):** Incorrectly predicted as positive. **FN (False Negatives):** Incorrectly predicted as negative.

Accuracy measures the proportion of correct predictions. However, it can be misleading on imbalanced datasets, as a model may appear accurate simply by predicting the majority class. In such cases, other metrics such as precision or recall may provide more insight.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision reflects the quality of positive predictions. It measures the proportion of true positive predictions among all positive predictions made by the model.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Recall, is a score of how well the model identifies actual positive cases. In applications like waste classification, high recall ensures important categories are not missed.

$$\text{F1 Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1 score is the harmonic mean of precision and recall. It provides a balanced metric that is especially useful when both false positives and false negatives carry significant consequences(Kashyap 2024).
After running my CNN model with PyTorch, these are the final scores that were collected.

Table 1: Classification Report by Class

Class	**Precision**	**Recall**	**F1-score**	**Support**
Cardboard	0.93	0.78	0.85	81
Glass	0.59	0.83	0.69	100
Metal	0.67	0.49	0.56	82
Paper	0.70	0.94	0.80	100
Plastic	0.77	0.60	0.67	97
Misc. Trash	1.00	0.33	0.50	27
Accuracy		0.71 (on 487 samples)		
Macro avg	0.78	0.66	0.68	487
Weighted avg	0.74	0.71	0.70	487

Although the model performs reasonably across most classes, cardboard stands out as the best accuracy for any category. Cardboard achieved a precision of 0.93, a recall of 0.78 and the highest F1-score of 0.85. This shows that the model consistently identifies cardboard correctly, both in terms of avoiding false positives and capturing most true positives.

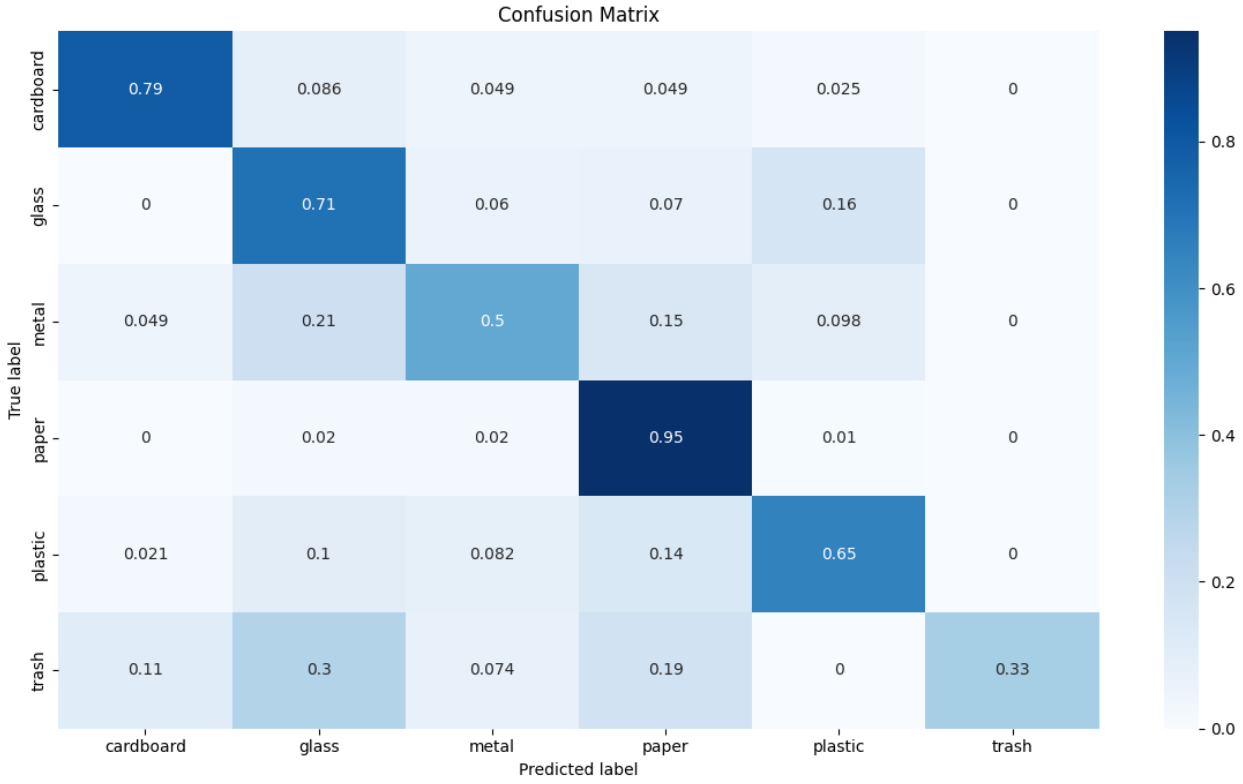

Figure 5: Confusion matrix for CNN

This is the confusion matrix, which shows how the model performed when predicting the data. As seen in Figure 2, paper and cardboard stand out as the best-recognised categories because they both have sharp edges and uniform colours, meaning it is easier to learn the spatial features, leading to better accuracy.

Let's compare this to the performance of the mlp:

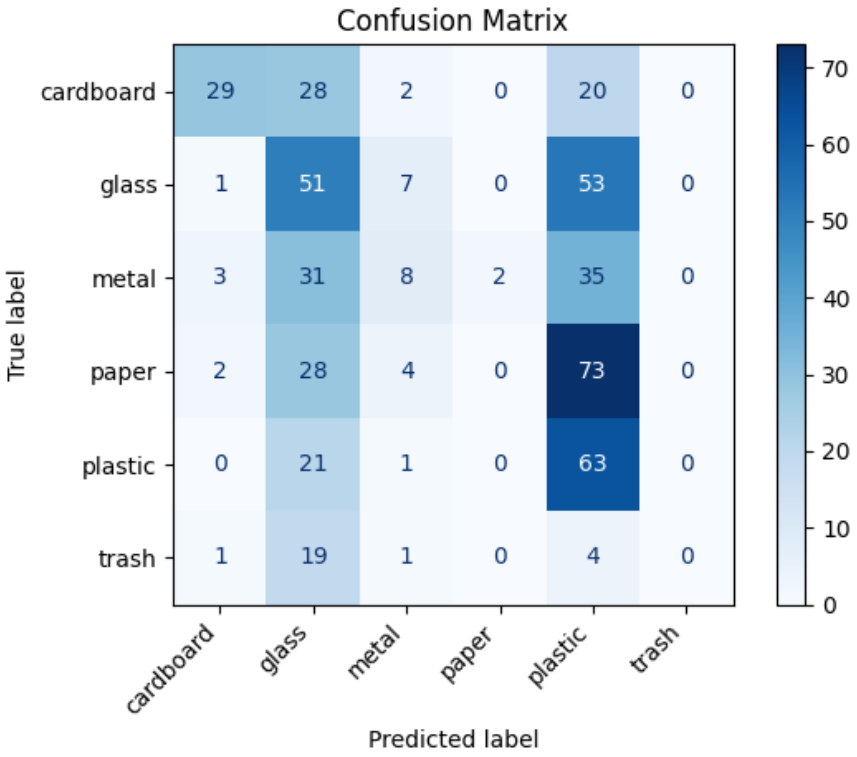

Figure 6: Confusion matrix for MLP

The model shows moderate overall accuracy at around 32%, indicating it can correctly classify some garbage types but struggles with many. It performs well on plastic with over 74% accuracy, likely due to distinct visual features and sufficient training samples. However, it fails completely to recognize paper and trash, highlighting issues with class imbalance or feature confusion. Overall, the model tends to favour more common or visually distinct classes while neglecting under-represented ones.

Evaluation

Throughout the course of this project, I have deepened my mathematical understanding of how Artificial Intelligence works and more specifically how convolutional neural networks can be applied to solve real-world classification tasks. I started this project with limited knowledge on machine learning and I finished with a working image classification model that was able to sort rubbish to a reasonable degree of accuracy.
At the start of this project, I believed that a simple multi-layer perceptron was sufficient to handle image classification for this task, but after building and testing the model it became clear that other methods needed to be employed. Although my initial belief was that the classes had very distinct features and colours where even a simple model could output some usable results; I realised that the images had only a few distinctive elements that were not sufficient to lead to promising results. I did not need to spend as much time building an MLP as I could have tested an pytorch implementation to test whether the model could

handle the task. This would have meant that I could spend more time working on the next steps of my project. In addition, I the results could have been improved if the chosen dataset had more images per class. This would mean that the model would have more training data, leading to better results and higher F1-score.

While the model does very well when it comes to predicting pictures of paper or cardboard, it struggles quite considerably when it comes to handling images with more ambiguous patterns or visually similar items such as plastic and glass. One possible improvement that I could implement would be incorporating augmentation techniques such as rotators, flips, and brightness shift to further increase the robustness of the model. Another way of improving model accuracy going forward would be to transfer learning using pre-trained models such as ResNet (He et al. 2016) or MobileNet (Howard et al. 2017).

Looking ahead, I hope to expand this project by integrating the model with real-time hardware using a Raspberry Pi connected to a camera system to allow automatic sorting in a physical environment.

References

- asdasdasasdas (2022), 'Waste classification data', `https://www.kaggle.com/datasets/asdasdasasdas/garbage-classification`.

Caiseri, E. (2022), 'Building a multi-layer perceptron from scratch with numpy', `https://elcaiseri.medium.com/building-a-multi-layer-perceptron-from-scratch-with-numpy-e4cee82ab06d`.

Goodfellow, I., Bengio, Y. & Courville, A. (2016), *Deep Learning*, MIT Press.

He, K., Zhang, X., Ren, S. & Sun, J. (2016), Deep residual learning for image recognition, *in* 'Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)', pp. 770–778.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M. & Adam, H. (2017), Mobilenets: Efficient convolutional neural networks for mobile vision applications.

Huang, G., Liu, Z., Van Der Maaten, L. & Weinberger, K. Q. (2017), 'Densely connected convolutional networks', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* pp. 4700–4708.

Kashyap, P. (2024), 'Understanding precision, recall, and f1 score metrics', `https://medium.com/@piyushkashyap045/understanding-precision-recall-and-f1-score-metrics-ea219b908093`.

Krizhevsky, A., Sutskever, I. & Hinton, G. E. (2012), Imagenet classification with deep convolutional neural networks, *in* 'Advances in Neural Information Processing Systems (NeurIPS)', pp. 1097–1105.

LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. (1998), 'Gradient-based learning applied to document recognition', *Proceedings of the IEEE* **86**(11), 2278–2324.

Nair, V. & Hinton, G. E. (2010), Rectified linear units improve restricted boltzmann machines, *in* 'Proceedings of the 27th International Conference on Machine Learning (ICML-10)', pp. 807–814.

Papers With Code (2023), 'Image classification on cifar-10', `https://paperswithcode.com/sota/image-classification-on-cifar-10`.

Powers, D. M. W. (2011), 'Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation', *Journal of Machine Learning Technologies* **2**(1), 37–63.

PyTorch Contributors (2019), 'Pytorch: An imperative style, high-performance deep learning library', `https://pytorch.org/`.

ResearchGate (2024), 'Scientific figure on researchgate: Multi-layer perceptron (mlp) basic architecture', `https://www.researchgate.net/figure/Multi-layer-perceptron-MLP-NN-basic-Architecture_fig2_354817375`.

siouladiv (2024), 'Porject source code', `https://github.com/siouladiv/EPQ-ML-classification`.

Tan, M. & Le, Q. (2019), Efficientnet: Rethinking model scaling for convolutional neural networks, *in* 'Proceedings of the International Conference on Machine Learning (ICML)'.